

平台 core 接口与 IMAS 兼容层

模块键：core 分组：00 平台核心 状态：接口层

主程序：modules/core/fusion_interface.py 默认输出目录：runs/interfaces/core

1. 模块定位

核心接口、模块规格、Plasma State 和 IMAS reduced/native 写出兼容层。

供所有模块调用，不是独立物理求解器。

平台核心接口层，负责模块规格、组件生命周期、PlasmaState 索引和 IMAS 兼容写出。

2. 输入

下表同时列出 interface 文件槽和界面/CLI 参数。interface 模式只显示模块声明的接口文件；manual 模式使用这些默认参数构造解析或 reduced 初态。

键	界面含义	模式	说明	默认
无文件输入	本模块从手动参数或结果目录扫描启动	可独立运行	不强制读取上游状态文件	

3. 输出

输出项	文件名	说明
manifest	imas_manifest.json	由模块运行后写入输出目录
plasma_state	plasma_state.json	由模块运行后写入输出目录
manual	core_manual.pdf	本 PDF 使用说明书

4. 物理公式与数值模型

本节写的是当前程序实际采用的 reduced 公式。符号约定：rho 为归一化半径， ΔV 为环向壳层体积，所有归一化量在写入 CSV/NC 前会按模块自己的标度转回工程量。

- (1)
 $FusionModuleSpec = key, title, inputs, outputs, upstream, downstream, reference_{codes}$
- (2)
 $FusionComponent. init/step/finalizedefinelifecyclehooks$
- (3)
 $PlasmaState = case_{name}, time_s, scalars, profiles, fields, distributions, diagnostics, provenance$
- (4)
 $state_m \mapsto \{IDS\}_m$
- (5)
 $reducedpayload = zlib(base64(JSON(numpyarrays)))withXIRONG_REDUCED_STATE_V1prefix$
- (6)
 $write_module_imas_statealwayswritesimas_{manifest}.json; native.ncisbest - effort$

5. 详细算法流程

本节按 平台 core 接口与 IMAS 兼容层

的源码入口、接口文件、核心函数、变量更新和输出文件逐步展开。这些步骤来自模块目录中的 Python 实现和 module_spec，优先描述本模块真实执行路径。

5.1 公共组件接口模型

- 1.1. 'modules/core/fusion_interface.py' 定义 'FusionPort'，字段是 'name'、'description'、'units'，用于描述模块输入输出端口。
- 1.2. 'FusionModuleSpec' 保存 'key'、'group'、'title'、'status'、'directory'、'purpose'、'inputs'、'outputs'、'config_keys'、'upstream'、'downstream' 和 'reference_codes'。
- 1.3. 'FusionRunResult' 保存组件生命周期返回的 'ok'、'message'、'outdir' 和 'artifacts'。
- 1.4. 'FusionComponent' 是基类，'init()' 默认成功，'step()' 默认返回 "interface only, solver not connected"，'finalize()' 默认成功。
- 1.5. 'PlaceholderComponent' 只把传入的 'FusionModuleSpec' 存为 'self.spec'，供所有 'modules/*/interface.py' 在真实求解器未接入时使用。
- 1.6. 'port(name, description, units="")' 是简化构造函数，源码中所有 spec 端口都通过它声明。
- 1.7. 'FusionRunResult.artifacts' 默认是 'field(default_factory=dict)'，避免多个运行结果共享同一个字典。
- 1.8. 'FusionComponent.__init__()' 把空配置规范化为 '{}', 所以 placeholder 组件总能读取 'self.config'。
- 1.9. 'FusionComponent.init/finalize' 使用 'self.spec.title' 生成消息，要求子类或 Placeholder 必须先设置 'spec'。
- 1.10. 默认 'step()' 返回 'ok=False'，这是接口未接真实求解器时的显式失败信号。

5.2 模块 spec 注册与查询

- 2.1. 'FUSION_MODULE_SPECS' 是 'FusionModuleSpec' 元组，集中登记 'device_machine'、'data_model'、'equilibrium_mhd'、'mhd_stability'、'core_transport' 等平台模块。
- 2.2. 每个 spec 明确写 'upstream' 和 'downstream'，例如 'data_model' 下游是 'workflow_coupling'、'visualization'、'validation_uq'。
- 2.3. 文件底部构造 'SPEC_BY_KEY={spec.key: spec for spec in FUSION_MODULE_SPECS}'，避免每次线性扫描。
- 2.4. 'get_spec(key)' 从 'SPEC_BY_KEY' 查找模块规格，若 key 不存在会抛出 'KeyError(f'Unknown module key: {key}'))'。
- 2.5. 'build_placeholder_component(key, config)' 调用 'get_spec(key)' 后返回 'PlaceholderComponent(spec, config)'。
- 2.6. 各模块 'interface.py' 的 'SPEC=get_spec('<module_key>')' 依赖这里的注册表，因此新增模块必须同时维护 core spec 或本地 desktop spec。
- 2.7. 'FusionModuleSpec.inputs/outputs' 默认空 tuple，避免可变默认端口列表。
- 2.8. 'reference_codes' 字段在 spec 中记录 EFIT、CHEASE、TRANSP、IMAS 等参考代码名，只作为文档/界面信息。
- 2.9. 'get_spec()' 是各 'modules/*/interface.py' 的公共入口，若 core spec 漏注册会在 import interface 时暴露错误。
- 2.10. 'build_placeholder_component()' 不检查模块状态是否可运行，只按 key 返回 placeholder。

5.3 PlasmaState 轻量索引

- 3.1. 'modules/core/plasma_state.py' 的 'PlasmaState' 是 JSON 可序列化 dataclass，字段包括 'case_name'、'time_s'、'scalars'、'profiles'、'fields'、'distributions'、'diagnostics' 和 'provenance'。
- 3.2. 'profiles'、'fields'、'distributions'、'diagnostics' 都是 'dict[str, str]'，保存 IDS URI 或文件路径，不保存大数组。
- 3.3. 'to_dict()' 直接调用 'asdict(self)'，保证嵌套 dict 和 list 能被 JSON 写出。
- 3.4. 'from_dict(cls, data)' 用 'cls(**data)' 复原状态索引，要求 JSON 字段名与 dataclass 字段一致。
- 3.5. 'save_json(path)' 先 'path.parent.mkdir(parents=True, exist_ok=True)'，再写 'json.dumps(..., indent=2, ensure_ascii=False)'。
- 3.6. 'load_json(path)' 读取文本后 'json.loads()'，再交给 'from_dict()'，供 data_model 或其它模块复用。

- 3.7. `PlasmaState.scalars`` 的类型标注允许 `float`、`str`、`bool`，但 `data_model` 实际也写入 `int`；Python `dataclass` 不在运行时强制类型。
- 3.8. `profiles/fields/distributions/diagnostics`` 存的是产品名到 URI 的映射，不包含 IDS 名称；IDS 名称保存在 `data_model` 的 `catalog CSV`。
- 3.9. `load_json()` 使用 `path.read_text()` 后 `json.loads()`，如果文件不存在或 JSON 损坏会直接抛异常。
- 3.10. `save_json()` 不返回路径，调用方需要自行记录 `state_path``，`data_model` 就是这样写入 `run_manifest``。

5.4 IMAS 写出入口

- 4.1. `modules/core/imas_compat.py`` 的 `write_module_imas_state(outdir,module_key,states,native_filename=None)`` 是 `device_machine` 和 `data_model` 使用的主写出函数。
- 4.2. 函数先把 `outdir`` 转为 `Path`` 并创建目录，然后将 `states`` 中每个 `state product`` 的值统一转换成 `np.asarray(value)``。
- 4.3. `state_product_name(name)`` 规范化产品名，`StateProduct(path, ids, keys)`` 记录目标 `.nc`` 路径、映射 IDS 和 `state keys``。
- 4.4. `product_ids(Path(name), module_key)`` 推断每个产品对应的 IDS 列表，随后合并去重；若没有 `workflow``，函数会追加 `workflow``。
- 4.5. `native_filename`` 为空时默认 `imas_{module_key}.nc``，否则使用调用方传入的 `device_machine_state.nc`` 或 `plasma_state_index.nc``。
- 4.6. `notes`` 固定说明 IMAS `.nc`` 是主外部状态文件，`reduced arrays`` 会嵌入 `workflow IDS payload``。
- 4.7. `normalized`` 字典中的每个 `state key`` 都被 `np.asarray`` 包装，字符串会变成 0 维 `numpy`` 字符串数组。
- 4.8. `products`` 中 `path`` 统一为 `outdir / f{name}.nc``，即使 `native_filename`` 指向单一 `.nc``，`manifest`` 仍记录每个产品的逻辑 `.nc`` 路径。
- 4.9. `ids = tuple(dict.fromkeys(...))`` 利用 `dict`` 保序去重，保持产品 IDS 的首次出现顺序。
- 4.10. 如果产品自身没有 `workflow IDS``，函数追加 `workflow``，保证 `reduced payload`` 至少能落入 `workflow comment``。

5.5 IMAS 原生导出流程

- 5.1. `write_module_imas_state()` 先调用 `imas_import_status()` 判断 `imas-python`` 是否可用。
- 5.2. 若可用，函数尝试 `export_state_products_to_imas(normalized,native_uri,module_key)``；成功时 `native_status='written``。
- 5.3. 导出失败时捕获异常，把状态写为 `export_failed: <type>: <message>``，追加 `Native IMAS export failed.``，并尝试删除残留 `.nc``。
- 5.4. 若 `imas-python`` 不可用，`native_status='imas_python_not_installed``，`manifest`` 仍写出目标 URI 和产品 `metadata``。
- 5.5. `export_state_products_to_imas()` 内部创建 `imas.IDSFactory()`，根据产品 IDS 名称创建 IDS 对象。
- 5.6. 每个 IDS 对象先 `_set_ids_comment()` 写 `encoded reduced payload``，再 `_fill_ids_from_state()` 尽可能填充标准 IDS 字段，最后通过 `imas.DBEntry(str(uri),'w')`` `put()`。
- 5.7. `export_state_products_to_imas()` 中 `getattr(factory, ids_name, None)`` 找不到 IDS creator 时跳过该 IDS，不立即失败。
- 5.8. `workflow IDS`` 会无条件尝试创建；若 `factory`` 有 `workflow``，则把 `encoded payload`` 写入 `ids_properties.comment``。
- 5.9. `_safe_set(workflow,'time',_time_axis(loader))`` 给 `workflow IDS`` 写时间轴，时间轴来自 `reduced state`` 中可识别的 `time`` 字段。
- 5.10. `DBEntry(str(uri),'w')`` 以写模式打开目标 URI，循环 `dbentry.put(ids_obj)`` 写每个 IDS。

5.6 IMAS 读取与 reduced payload

- 6.1. `load_imas_state(uri,state_name=None)``、`load_imas_state_products(uri)`` 和 `read_standard_imas_state()` 提供从 `.nc`` 或 IMAS URI 回读 `reduced state`` 的能力。
- 6.2. `inspect_imas_entry(uri,ids_names=None)`` 用于列出 IDS 非空路径和可读状态，给 `data_model`` 的 `imas_bridge.py`` `inspect`` 使用。

- 6.3. ``_encode_state_payload(module_key, states)`` 把 reduced state 序列化进 workflow IDS comment，解决标准 IDS 字段不完整时的保底传输。
- 6.4. ``_decode_state_payload(comment)`` 反向解析 comment 中的 reduced JSON，``_encode_payload_value()`` 和 ``_decode_payload_value()`` 处理数组、标量和字符串类型。
- 6.5. ``_fill_core_profiles()``、``_fill_equilibrium()``、``_fill_waves()``、``_fill_core_sources()`` 等函数把 reduced key 映射到常见 IDS 字段。
- 6.6. ``_time_axis(loaded)``、``_first_array()``、``_profile_history()``、``_radialize()`` 等工具函数为不同状态形状生成统一时间轴和径向剖面。
- 6.7. ``read_imas_equilibrium_state()``、``read_imas_waves_state()`` 等函数按 IDS 类型提取常用数组，供后续模块读取标准 IMAS 数据。
- 6.8. ``_db_get_optional()`` 和 ``_ids_array()`` 封装安全读取，避免某些 IDS 字段缺失时整个读取流程崩溃。
- 6.9. ``_interp_like()``、``_axis_from_grid()`` 等工具把不同维度网格转换为 reduced 模块需要的一维剖面。
- 6.10. ``_ids_non_empty_paths()`` 用于 inspect 输出，帮助定位某个 IDS 中实际填充了哪些路径。

5.7 核心层与模块的边界

- 7.1. 核心层不直接运行物理求解；它提供 spec、placeholder 生命周期、PlasmaState JSON 和 IMAS 兼容读写。
- 7.2. 所有本轮负责模块的 ``interface.py`` 都只调用 ``build_placeholder_component()``，所以详细算法流程必须转向各模块脚本，而不是虚构 ``FusionComponent.step()``。
- 7.3. 当模块调用 ``write_module_imas_state()`` 时，核心层负责写 ``imas_manifest.json`` 和可选 ``.nc``，模块负责提供 state payload 和 CSV/PNG。
- 7.4. 当模块使用 reduced manifest 而不是 ``write_module_imas_state()`` 时，核心层只提供 plotting 或 stage7 CSV 工具，不承担 IDS 投影。
- 7.5. ``PlasmaState`` 的设计意图是保存 metadata 和 URI，大数据组应留在模块 IMAS netCDF 或 CSV 中。
- 7.6. PDF 中的 core 算法页应重点解释公共契约如何串联 module spec、interface、状态索引和 IMAS manifest，而不是描述某个单独物理模型。
- 7.7. 核心层 ``imas_compat`` 会在 manifest 中写 adapter ``xirong-imas-compat`` 和 ``ADAPTER_VERSION``，用于区分 reduced manifest 手写版本。
- 7.8. 使用 reduced manifest 的模块自行实现 ``write_reduced_manifest()``，其 schema 和字段与核心 ``IMASManifest`` 不完全相同。
- 7.9. core spec 和 module-local ``*_module_spec.py`` 并存：前者服务统一接口，后者服务桌面参数和 PDF 生成。
- 7.10. 本 JSON 的 core 流程应覆盖公共 API、数据容器和 IMAS 适配器三条路径，因为 core 目录没有单一可运行 CLI。

6. 使用说明

在主桌面选择本模块后点击“使用说明”会打开本 PDF；在模块独立启动界面中点击同名按钮也会打开同一个 PDF。

推荐流程：先按默认 manual 参数运行一次，检查 final_summary.csv 和预览图；再切到 interface 模式接入上游 .nc/CSV；最后用输出目录中的 CSV/NC 做下游耦合或报告。

如果 interface 文件不存在，部分 reduced 原型会回退到解析默认剖面；工程校核时应在 final_summary.csv 或 imas_manifest.json 中确认实际读取来源。

7. 假设、边界和风险

- core 本身不定义物理求解公式。
- IMAS 字段映射是 best-effort，Data Dictionary 差异可能导致 native export 失败。
- 无 IMAS-Python 时只有 manifest/reduced 产物。

8. 维护要求

当 module_spec、CLI 参数、输出文件或核心公式变化时，必须重新运行 scripts/docs/build_module_manuals.py 生成本 PDF，避免界面、脚本和说明书口径不一致。